

3. An appropriate machine learning algorithm

In this article, we will use a case study to show how you can get started with NLP and ML. The example used below is super simple and easy to follow. But before we get started with the case study, let me introduce you to the Multinomial Naïve Bayes algorithm that we shall be using to build our machine learning model.

The Multinomial Naïve Bayes algorithm

The Naïve Bayes algorithm is a probabilistic classifier used for predictive analysis. It is simpler as compared to other algorithms and has been known to have a higher success rate. Naïve Bayes makes the assumption that all input attributes are conditionally independent. It is highly scalable and works on the principle of learning by doing.

It takes preprocessed data with the extracted features required as input for training. Once trained, it can be used to provide polarity of a given input text, i.e., if the text is positive, negative or neutral.

Bayes theorem states that the posterior probability $P(A|B)$ is calculated from $P(A)$, $P(B)$, and $P(B|A)$. Naive Bayes classifiers assume that the effect of the value of a predictor (y) on a given class (X) is independent of the values of other predictors. What makes Naïve Bayes distinct from the more traditional Bayes Belief Networks (BBN) is that it assumes complete conditional independence of all input attributes. The Bayes theorem is given as:

$$P(A/B) = \frac{P(B/A) \cdot P(A)}{P(B)}$$

where A and B are two events that are independent of each other and make an equal contribution to the outcome.

If we generalise the Naïve Bayes algorithm, we get:

$$P(y/X) = \frac{P(X/y) \cdot P(y)}{P(X)}$$

where y is the class variable and X is the feature vector of size 'n', i.e., $X = (x_1, x_2, x_3, \dots, x_n)$.

In sentiment analysis, for certain cases, finding the word frequency or discrete count can be beneficial in increasing the accuracy of the machine learning model. In such cases, Multinomial Naïve Bayes, a variant of the standard Naïve Bayes can be used. In MNB, the assumption is that the distribution of each feature, i.e., $P(f_i|C)$, is a multinomial distribution.

Multinomial distribution is defined as:

$$P = \frac{n!}{(n_1!)(n_2!)\dots(n_x!)} P_1^{n_1} P_2^{n_2} \dots P_x^{n_x}$$

where n is the total number of events, $n_1 \dots n_x$ are the number of outcomes for each event, and $P_1 \dots P_x$ are the probabilities of the occurrence of each event.

Case study: Sentiment analysis of statements made in 'finance' related news using the Multinomial Naïve Bayes algorithm

For the purpose of this case study, I have made use of a data set that is freely available on Kaggle. This is a simple data set that is extremely ideal for beginners who are just getting started with sentiment analysis. It contains two features, namely, the sentences and their corresponding sentiments. The sentiment for each sentence can either be positive, negative or neutral. This data set contains 5322 unique sentences, which are plenty for training and testing our algorithm.

You can download the data set for free from <https://www.kaggle.com/code/turankeskin/financialsenti/data>. Now place it in the same folder as your notebook or Python file to make things easier.

Step 1: We will start by importing the Python libraries necessary for this project:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import re
import string
import nltk

nltk.download('stopwords')
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')

from wordcloud import WordCloud

from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer, PorterStemmer
from nltk import pos_tag, ne_chunk
from nltk.chunk import tree2conlltags

import warnings
warnings.filterwarnings("ignore")
```

Step 2: Using pandas, we will read the csv file and create an instance:

```
Data = pd.read_csv("/kaggle/input/financial-sentiment-analysis/
data.csv")
df = data.copy()
```

Step 3: To get an idea of how many positive, negative and neutral sentences we have in the data set, we are going to plot